

Zadanie 2

Otwarto: wtorek, 22 kwietnia 2025, 12:00

Wymagane do: niedziela, 18 maja 2025, 23:59

Źródło: MIMoodle – platforma Moodle Wydziału Matematyki, Informatyki i Mechaniki Uniwersytetu Warszawskiego.
Treść zadania przygotowali prowadzący przedmiot.
moodle.mimuw.edu.pl/mod/assign/view.php?id=149773

Pierwiastek kwadratowy

Dla danej nieujemnej $2n$ -bitowej liczby całkowitej X chcemy wyznaczyć taką nieujemną n -bitową liczbę całkowitą Q , że $Q^2 \leq X < (Q + 1)^2$.

Polecenie

Zaimplementuj w assemblerze wołaną z języka C funkcję o następującej deklaracji:

```
void nsqrt(uint64_t *Q, uint64_t *X, unsigned n);
```

Parametry Q i X są wskaźnikami na binarną reprezentację odpowiednio liczb Q i X . Liczby są zapisywane w naturalnym kodzie binarnym, w porządku cienkokońcówkowym, po 64 bity w każdym słowie typu `uint64_t`. Parametr n zawiera liczbę bitów i jest to liczba podzielna przez 64 z przedziału od 64 do 256000.

Pamięć wskazywana przez X jest modyfikowalna z intencją używania jej jako pamięci roboczej.

Sugerowany algorytm

Wynik obliczamy iteracyjnie. Niech $Q_j = \sum_{i=1}^j q_i 2^{n-i}$, gdzie $q_i \in \{0, 1\}$, będzie wynikiem po j iteracjach, a R_j resztą po j iteracjach. Przyjmujemy $Q_0 = 0$ i $R_0 = X$. W iteracji j wyznaczamy bit q_j wyniku. Niech $T_{j-1} = 2^{n-j+1}Q_{j-1} + 4^{n-j}$ dla $j = 1, 2, \dots, n$. Jeśli $R_{j-1} \geq T_{j-1}$, to przyjmujemy $q_j = 1$ i $R_j = R_{j-1} - T_{j-1}$, a w przeciwnym przypadku $q_j = 0$ i $R_j = R_{j-1}$. Zachodzi zależność rekurencyjna $R_j = R_{j-1} - q_j(2^{n-j+1}Q_{j-1} + 4^{n-j})$, z której wynika, że po n iteracjach mamy $R_n = X - Q_n^2$. Jako ćwiczenie proszę udowodnić, że $0 \leq R_n \leq 2Q_n$.

Oddawanie rozwiązania

Jako rozwiązanie należy wstawić w Moodle plik o nazwie `nsqrt.asm`.

Kompilowanie

Rozwiązanie będzie kompilowane poleceniem:

```
nasm -f elf64 -w+all -w+error -o nsqrt.o nsqrt.asm
```

Rozwiązanie musi się kompilować i działać w laboratorium komputerowym.

Przykłady użycia

Przykłady użycia znajdują się w niżej załączonych plikach `nsqrt_example.c` i `nsqrt_example.cpp`. Kompiluje i łączy się je z rozwiązaniem poleceniami:

```
gcc -c -Wall -Wextra -std=c17 -O2 -o nsqrt_example_64.o nsqrt_example.c
gcc -z noexecstack -o nsqrt_example_64 nsqrt_example_64.o nsqrt.o
g++ -c -Wall -Wextra -std=c++20 -O2 -o nsqrt_example_cpp.o nsqrt_example.cpp
g++ -z noexecstack -o nsqrt_example_cpp nsqrt_example_cpp.o nsqrt.o -lgmp
```

Ocenianie

Ocena składa się z dwóch części.

1. Zgodność rozwiązania ze specyfikacją będzie oceniana za pomocą testów automatycznych, za które dostaje się maksymalnie 7 punktów. Oprócz poprawności wyniku sprawdzane będą przestrzeganie reguł ABI, poprawność odwołań do pamięci i zajętość pamięci, w tym również stosu. Na ocenę będą miały wpływ rozmiary sekcji programu i szybkość jego działania. Za błędną nazwę pliku odejmiemy jeden punkt.
2. Za formatowanie i jakość kodu dostaje się maksymalnie 3 punkty. Tradycyjne formatowanie programów w assemblerze polega na rozpoczynaniu etykiet od pierwszej kolumny, a mnemoników rozkazów od wybranej ustalonej kolumny. Nie stosuje się innych wcięć. Taki format mają przykłady pokazywane na zajęciach. Kod powinien być dobrze skomentowany, co oznacza między innymi, że każdy blok kodu powinien być opatrzony informacją, co robi. Należy opisać przeznaczenie rejestrów. Komentarza wymagają wszystkie kluczowe lub nietrywialne linie kodu. W przypadku assemblera nie jest przesadą komentowanie prawie każdej linii kodu, ale należy unikać komentarzy opisujących to, co widać.

Zastrzegamy sobie uzależnienie wystawienia oceny od osobistego wyjaśnienia szczegółów działania programu prowadzącemu zajęcia.

Załączniki

- `nsqrt_example.c` 13 kwietnia 2025, 19:34
- `nsqrt_example.cpp` 22 kwietnia 2025, 12:10